

# Safe scripting

Scripts that delete files or stop builds can hurt when they run by accident

# Fail fast, quote, dry-run

- Set dash-euo pipefail at the top
- Always quote expansions like dollar-file so spaces do not split paths
- For deletes and cleans
- Check that directories exist before you walk them

# Browser lab

Digital Design and Verification Platform

HomeTools

HomeToolsSafe scripting

Practice `set -euo pipefail`, quoting, and `--dry-run` habits before a script touches real files.

Starter example

A clean script with `set -euo pipefail`, quoted vars, and `--dry-run`. Toggle flags / scenarios to see what breaks without each habit.

Load starter example

Challenges 0 / 32 cleared

Quiz: `set -e` aborts when a command? Answer: `false`

Answer

Show hintCheckNextIdle

Quiz: `set -e` | Quiz: `set -u` | Quiz: `pipefail` | Run starter | Missing `set -e` | Unbound abort | Pipe masked | Pipefail catches | Unquoted split | Quoted ok | Dry-run msg | Quiz: quote | Quiz: dry-run | Scan full | Quiz: shebang | Combo line

Quiz: `$(!)` | Real delete | Score 6/6 | Quiz: order | Toggle off `set -e` | Quiz: why

Script + flags

Safe clean script | Missing `set -e` | Unbound variable

Pipe without `pipefail` | Pipe with `pipefail` | Unquoted path

Quoted path | Dry-run clean

☒ `set -e` ☒ `set -u` ☒ `set -o pipefail`

Args (space-separated)

--dry-run

```
#!/usr/bin/env bash
set -euo pipefail

DRY_RUN=0
if [[ "${1:-}" == "--dry-run" ]]; then
  DRY_RUN=1
fi

set -o errexit
set -o nounset
set -o pipefail

build
```

Run scriptScan checklist

Checklist

6 / 6

☒ Shebang present (`#!/usr/bin/env bash`) - detected

☒ `set -e` (abort on failing command) - detected

☒ `set -u` (abort on unbound variable) - detected

☒ `set -o pipefail` (fail if any pipe stage fails) - detected

☒ Quote expansions: `"$var"` not `$var` - detected

☒ Support `--dry-run` for destructive actions - detected

Trace

```
#!/usr/bin/env bash
set -euo pipefail
DRY_RUN=0
parse_args() {
  while [[ $# -gt 0 ]]; do
    case "$1" in
      --dry-run)
        DRY_RUN=1
        ;;
      *)
        echo "Error: Unknown option '$1'"
        exit 1
      ;;
    esac
    shift
  done
}

parse_args "$@"

if [[ "$DRY_RUN" == "1" ]]; then
  echo "Dry-run mode enabled. No real actions will be performed."
  exit 0
fi

build
```

Habits

set -euo pipefail

Fail fast on errors, unbound vars, and failed pipe stages.

Quote expansions

Use `"$FILE"` so spaces don't split into many words.

--dry-run

Print what would happen before deleting or overwriting.

Bad: is `FFILE` with spaces - two arguments

Good: is `"FFILE"` - one path

Cheat sheet

| Flag / habit                 | What it prevents                             |
|------------------------------|----------------------------------------------|
| <code>set -e</code>          | Continuing after a failed command            |
| <code>set -u</code>          | Silent empty expansion of types / unset vars |
| <code>set -o pipefail</code> | Ignoring a failing left side of a pipe       |
| <code>"\$var"</code>         | Word-splitting and globbing surprises        |
| <code>--dry-run</code>       | Accidental destructive actions               |

- Prefer `$(!)` when reading optional args under `set -u`.
- Scan - fix - run with `--dry-run` before a real clean.

## Safe scripting lab

learn\_unix — safe scripting

# Real shell practice

```
wsl - module18-safe-scripting/examples/safe_scripting

# Track A - real Unix
# real shell session (Track A)

$ ls tmp/
cache1.tmp
cache2.tmp

$ chmod u+x clean_tmp.sh

$ ./clean_tmp.sh
Would delete: tmp/cache1.tmp (run with --confirm to actually delete)
Would delete: tmp/cache2.tmp (run with --confirm to actually delete)

$ ls tmp/
cache1.tmp
cache2.tmp
```

Real shell — dry-run clean

## Real shell practice — try these

```
# ls tmp/ — see the sample cache files before cleaning
ls tmp/
```

```
# chmod u+x clean_tmp.sh — make the clean script executable
chmod u+x clean_tmp.sh
```

```
# ./clean_tmp.sh — dry run by default (prints Would delete, keeps
files)
./clean_tmp.sh
```

```
# ls tmp/ — confirm the files are still there
ls tmp/
```

```
# ./clean_tmp.sh --confirm — only when ready: actually delete *.tmp
# ./clean_tmp.sh --confirm
```

# Pitfalls to watch

- Unquoted variables break on spaces and can make `rm` touch the wrong paths
- Skipping dry-run on a clean script is how accidents happen
- And remember

# Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, finish a few challenges after the starter
- On the real shell, run the dry-run clean and only use confirm when you intend to delete
- When you are ready, take the short quiz, then continue to project layout, archives, sed